

NAME : DISHA N. SHIRASKAR

ENTRY NO : 2021CS10578

COL226 ASSIGNMENT 1.

classmate

Date _____

Page _____

To Prove : The long division method correctly gives us the integer square root of a given number 'a' and also the least number which should be subtracted from 'a' to get a perfect square.

Proof : By Logical Deduction;

Assume :

'a' is the given number whose square root has to be found.

'x' be the value of integer square root of 'a'

'y' be the least significant digit of x

'z' be the number obtained after subtracting y from x.

From our assumptions we have, $x = 10z + y$.

During every recursive step of long division, the value of a is updated; let us call it to be "current dividend" a. At the first step, the current dividend is either { first 2 digits or first 1 digit of the given no. 'a'.

For subsequent iterations :

current dividend = $(100) * (\text{remainder from previous call}) + \text{next two digits of a}$.

During each iteration, our goal is to find 'y' such that

$$(2 \cdot 10z + y)^2 \leq a \quad \& \quad (10z + y + 1)^2 > a \quad \text{--- (3)}$$

The above 2 conditions also guarantee the uniqueness of y.

Simplifying the first inequality we get;

$$(2 \cdot 10z + y)y \leq a - (10z)^2 \quad \text{--- (4)}$$

This clearly explains the step of our long division method since we do subtraction from the leftmost digits of given number 'a'.

(Note that in (3) & (4) 'a' represents the current dividend).

Analyzing the LHS;

$2 \cdot 10z$ involves doubling the number z and multiplying further by 10 to make space for next digit y.

We have to choose the next digit y, so that the new number $(20z + y)$ when multiplied by y equals / less than equal to the number obtained after subtraction on RHS. This is exactly what we do in long division.

In this way, the process of long division helps us find next digit of running quotient, given current dividend & current quotient.

→ or when no. of digits of dividend is exhausted,

The process terminates when we are unable to find any y such that equations in (3) aren't satisfied, i.e. $(10z + y)^2 > a \quad \forall y \in [0, 9]$

The current quotient at this step is the integer square root of the given number.

Hence Proved.

NAME : DISHA N. SHIRASKAR

ENTRY NO : 2021CS10578

COL226 ASSIGNMENT 1.

classmate

Date _____

Page _____

Proof of correctness of my algo :

Step 1 : My algorithm first accepts the given number as a string ;

Then `make_str` function checks the no. of digits ~~are~~ if odd it adds an additional "zero" to the left of string else returns it as it is.

Step 2 : This modified string is given as input to function `make_list`

which pairs up the digits of string and inserts them in a list

& finally returns the list.

(This process is same as we make groups of 2 digits of a number before starting long division).

Step 3 : Then, helper function is called with arguments as first element of list,

(which is the current dividend), 0 (which is current quotient), 0 (which current divisor) ; 1 (length of list upto which long division has been carried out) and the `str_list`.

In the helper function, we find each digit of quotient by calling helper recursively. Initially, the helper finds the "y" such that conditions in (3) (in proof of correctness are satisfied).

after finding ; the current ~~and~~ dividend is updated (by subtracting $(10 * \text{curr_quo} + y)y$ from it and adding the next element of list) ;

`curr_quo` is updated by $10 * \text{curr_quo} + y$; divisor ~~to~~ is also updated ; length is incremented by 1 & helper is called again.

Helper is called ~~until~~ until we finish traversing the list completely, at the end of which helper returns the final remainder & final integer square root of given number as a tuple.

In this way, my algorithm executes all the steps similar to integer long division and I have already proved that integer long division evaluates square root of given number correctly. ; therefore my algorithm also evaluates integer square root correctly.

Hence Proved.

Since there is a limit to max integer in SML ; in order to deal with larger inputs ~~we~~ I have defined my own arithmetic operations (add, sub, mult, compare)

which perform operations digit by digit instead considering the entire integer at once.

NAME : DISHA N. SHIRASKAR

ENTRY NO : 2021CS10578

COL226 ASSIGNMENT 1

classmate

Date _____

Page _____

PSEUDO CODE :

fun make_list (string, list, index) :

Before calling the fn process the string such that it has even number of digits.

Append the first 2 digits of string into the list

if index = len(string) - 2 :

return list

else : make_list(string, list, index + 2).

fun int_sqrt (string) :

l = make_list (string, [], 0) ; x = first 2 digits of given string.

helper (x, 0, 0, 1, l).

fun helper (num1, num2, n, len, str_list) :

Here num1 : Current dividend.

num2 : Quotient ; n : Current divisor

len : length of str_list processed. ; str_list : list of digits of string in groups of 2.

if len = length(str_list) + 1 :

return (num1, num2)

else : find 'y' (a digit from 0 to 9) treating 'n' as 'z' and 'num1' as 'a' (Referring to proof of correctness of algo). using binary search.

if len = length(str_list) :

Indicating we are at last 2 digits of given number.

helper ((num1 - (y(10n + y))), 10 * num2 + y, 10n + h, len + 1, str_list)

else

val r = (num1 - (y(10n + y))) * 100 + next 2 element of str_list.

helper (r, 10 * num2 + y, 10n + h, len + 1, str_list).

#PSEUDOCODE (Helper functions)

(1) fun equal_len (num, req, curr) :

Takes a string as argument & makes it of required length.

if req == curr : return num

else : new_num = ("0" ^ num)

equal_len (new_num, req-1, curr).

(2) fun equal_len_final (num1, num2) :

Takes 2 strings as argument; makes both of them of equal length to perform binary operations on them. ; returns a tuple.

let : ~~var~~ s1 = size of string num1.

s2 = size of string num2.

in : if s1 > s2 : [num1, equal_len (num2, s1, s2)]

else : [equal_len (num1, s2, s1), num2]

(3) fun add (num1, num2, carry, index, sum).

Takes 2 strings to be added, carry, index upto which sum has been calculated and curr_sum as calculated as arguments).

if :

↳ sum_p = digit_1_i + digit_2_i + carry

next_carry = sum_p div 10

next_sum = str(sum_p mod 10) ^ sum

if you are at most significant index :

add (num1, num2, next_carry, i-1, str(sum_p) ^ sum)

↳ So that you don't lose last carry.

if index = -1

(# indicating both strings are processed completely)

return sum.

else : add (num1, num2, next_carry, i-1, next_sum).

(4) fun multi_10x (num, t)

Takes a string as arg. & concatenates "0" if t=1 & "00" if t=2 resembling to Int. multiplication by "10" & "100" resp.

if t=1 : return num ^ "0"

else : return num ^ "00"

(5) fun multi(num1, num2, carry, index, prod):

Takes 2 strings as arguments along with current carry, index & evaluated product string.

if index = -1 (entire string has been traversed) : return prod.

else : Take ith digits of num1 and num2 as integer

Find their product and next curr carry to it (new-prod)

next_carry = new_prod div 10

new_prodf = new_prod mod 10

if index = 0 (you are at most significant digit).

multi(num1, num2, next_carry, ^{index-1}str(new_prod) ^ prod)

else : multi(num1, num2, next_carry, index-1, new_prodf ^ prod)

(6) fun sub(num1, num2, carry, index, diff):

Takes 2 strings to be subtracted as arguments ; subtraction is same as adding 10's complement of num2 to num1 & ignoring the last carry.

make num1 and num2 of equal length using equal-len-final.

if index = -1 (All digits of both strings have been subtracted)

then return diff

else : Consider ith digit of num2 at index .

Find 9's complement of num2 as 9 - num2 = num2_f

diff_f = num2_f + num1_index + carry

new_carry = diff_f div 10

new_diff = diff_f mod 10

sub(num1, num2, new_carry, index-1, new_diff ^ diff)

(Note that to ensure the calculation of 10's complement of num2 we give carry = 1, when sub is called for the first time).

(7) fun compare (t, num1, num2, index).

t stands for type of comparison if $t=1$ it ^{does} greater than or equal to comp & if $t=2$ it does greater than comparison.

num1 and num2 are traversed from leftmost digit at any point
if left digit of num1 > num2 digit of num2 it returns true.
else if digit of num2 > num1 digit of num1 it returns false
else compare (t, num1, num2, i+1).

if index becomes equal to size of num1 or num2 (which means all digits are equal); then

if $t=1$: return true

else ~~t=2~~ : return false.

(Note that num1, num2 have equal length).

(8) fun compare_final (t, num1, num2) : makes $\pm$ num1 and num2 of equal len
& calls compare (t, num1, num2, 0).

(9) fun find (num1, num2, ^{low, high} ~~l, r~~, arr) :

This function helps us find the next digit of quotient using binary search in a list of numbers from 0 to 9. $l \rightarrow$ left index of arr; $r \rightarrow$ right index of arr.

mid = $(l+h)/2$

if ~~r-l~~ $= 1$ (means you have reached the final state where you're arr has only 2 nos).

f_low = $(10 \times \text{num1} + \text{arr}[\text{low}]) \times \text{arr}[\text{low}]$

f_high = $(10 \times \text{num1} + \text{arr}[\text{high}]) \times \text{arr}[\text{high}]$

if $f_low \leq \text{num2}$ & $f_high \leq \text{num2}$: return (arr[low], $10 \times \text{num1} + \text{arr}[\text{low}]$)
else : return (arr[high], $10 \times \text{num1} + \text{arr}[\text{high}]$)

else if : $((10 \times \text{num1} + \text{arr}[\text{mid}]) \times \text{arr}[\text{mid}]) < \text{num2}$

then find (num1, num2, mid, high, arr)

else

find (num1, num2, low, mid, arr).

(in above code $l = \text{low}$,
 $h = \text{high}$)